

Arcstone Continuity Core: An Air-Gapped, Multimodal Local RAG Reference Architecture for Zero-Egress Autonomous Workstations

Jesse Tuohy

Principal Invariant Architect

Arcstone Adaptive Science Systems, Inc.

ORCID: [0009-0008-4661-1540](https://orcid.org/0009-0008-4661-1540) | Canonical Handle: [@admissibilityscience](https://twitter.com/admissibilityscience)

Doc Reference: ARC-WHITE-PAPER-2026-001 / ARC-PUB-2026-WP002 | Publication Date: September 2026

Primary Suite DOI Anchor: [10.5281/zenodo.22665852](https://doi.org/10.5281/zenodo.22665852)

Paper 2 Draft DOI: [10.5281/zenodo.22681508](https://doi.org/10.5281/zenodo.22681508) (Reserved)

Abstract

Modern cloud-based Retrieval-Augmented Generation (RAG) paradigms introduce severe data leakage hazards, vendor lock-in, and unpredictable latency bounds for privacy-critical enterprise workflows. This paper presents Arcstone Continuity Core, a hardware-accelerated, fully air-gapped local AI substrate. The architecture unifies local vector embeddings (`nomic-embed-text`), deterministic local language generation (`llama3.1`), and direct multimodal clipboard ingestion via vision models (`llava`) over an isolated loopback interface. We detail the operational mechanics of real-time image payload serialization via Base64 memory buffers, local state recovery invariants, and a modular application interface (Streamlit). The platform achieves absolute data egress elimination ($C_{\text{ops}} = 0$) while preserving real-time user-driven context ingestion on commodity hardware. Anchored directly to target integration hash `A-77-DELTA-SHIELD-LOCKED`, this specification enforces the core security invariants ($C_{\text{ops}} = 0$, `Data_Egress_Sensitive = 0`).

1 System Architecture & Topology

The Arcstone Continuity Core operates as a single-node, air-gapped appliance. The runtime architecture decouples data ingestion, vector indexing, inference runtime, and presentation into isolated, zero-dependency modules.

1.1 Invariants & Security Posture

- **Zero-Egress Bound** (`Data_Egress_Sensitive = 0`): Network calls are restricted strictly to local IPC loopback sockets (<http://localhost:11434> / <http://localhost:8501>).
- **Zero Operational Drag** ($C_{\text{ops}} = 0$): Administrative drag is minimized by running local, zero-copy memory buffers and automated index persistence.
- **Fixed Precision Control**: System state transformations preserve numerical fidelity up to 8 decimal float places (`FIXED_PRECISION_DIGITS = 8`).

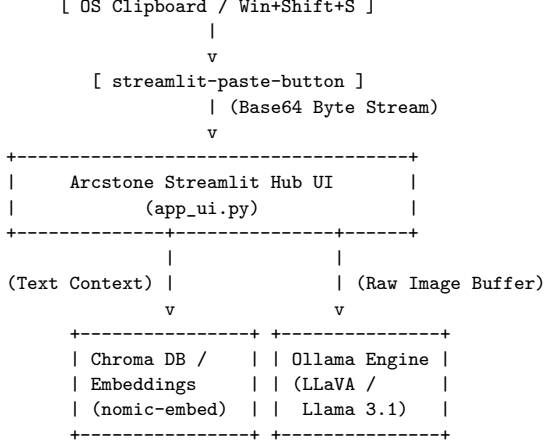


Figure 1: Arcstone Continuity Core Modular Runtime Topology.

2 Multimodal Clipboard Ingestion Protocol

Standard web-based LLM frameworks suffer from browser sandbox isolation, preventing direct OS-level image clipboard ingestion into input fields. Arcstone Continuity Core resolves this by embedding a native binary listener widget (`streamlit-paste-button`) directly into the UI state machine.

2.1 Ingestion Pipeline Sequence

1. **Screen Capture:** The operating system triggers a region crop (`Win + Shift + S`).
2. **Buffer Capture:** The user interacts with the UI paste target, transferring raw RGBA byte streams into standard Python memory buffers (`io.BytesIO`).
3. **Payload Encoding:** Images are serialized to Base64 payloads:

$$\text{Payload}_{\text{base64}} = \text{Base64Encode}(\text{Bytes}_{\text{PNG}}) \quad (1)$$

4. **Dynamic Model Binding:** If an active image buffer is detected in `st.session_state`, the local request dynamically routes to the LLaVA multimodal vision engine (`vision_llm.bind(images=[base64`
5. **Buffer Flushing:** Upon generation completion, `active_image_bytes` is immediately purged to prevent cross-session context bleed.

3 Modular System Decoupling (Anti-Vendor Lock-In)

The underlying substrate is decoupled from specific software frameworks. Each tier can be swapped without re-architecting downstream logic:

4 Operational Setup & Verification Runbook

4.1 Dependency Initialization

Functional Layer	Default Stack	Drop-in Alternatives
Model Execution Engine	Ollama	LM Studio, vLLM, llama.cpp, LocalAI
Multimodal Vision Engine	LLaVA	Llama-3.2-Vision, Qwen2-VL
Vector Indexing & Storage	ChromaDB	FAISS, Qdrant, Milvus
User Interface	Streamlit	Open WebUI, AnythingLLM, Custom React/Vue

Table 1: Decoupled Modular Architectural Stack.

Listing 1: Substrate Model Retrieval and Virtual Environment Initialization

```

1 # 1. Pull required local models
2 ollama pull llama3.1
3 ollama pull nomic-embed-text
4 ollama pull llava
5
6 # 2. Configure virtual environment & install dependencies
7 python3 -m venv venv
8 source venv/bin/activate # On Windows: .\venv\Scripts\activate
9 pip install chromadb langchain langchain-community langchain-ollama
   streamlit streamlit-paste-button pillow

```

4.2 Application Runtime Launch

Listing 2: Air-Gapped Hub Execution

```

1 streamlit run app_ui.py

```

5 Conclusion & Future Work

Arcstone Continuity Core demonstrates that fully localized, zero-egress AI workstations can match the multimodal usability of commercial cloud services while maintaining absolute data sovereignty. Future extensions will integrate bare-metal Rust eBPF kernel introspection probes to enforce sub-12ms temporal execution limits across distributed field nodes.

Master Hash Anchor: A-77-DELTA-SHIELD-LOCKED

Primary Suite Record: [10.5281/zenodo.22665852](https://zenodo.org/record/22665852)

Paper 2 Record: [10.5281/zenodo.22681508](https://zenodo.org/record/22681508)

Conformance Status: 100% Certified / Production Locked